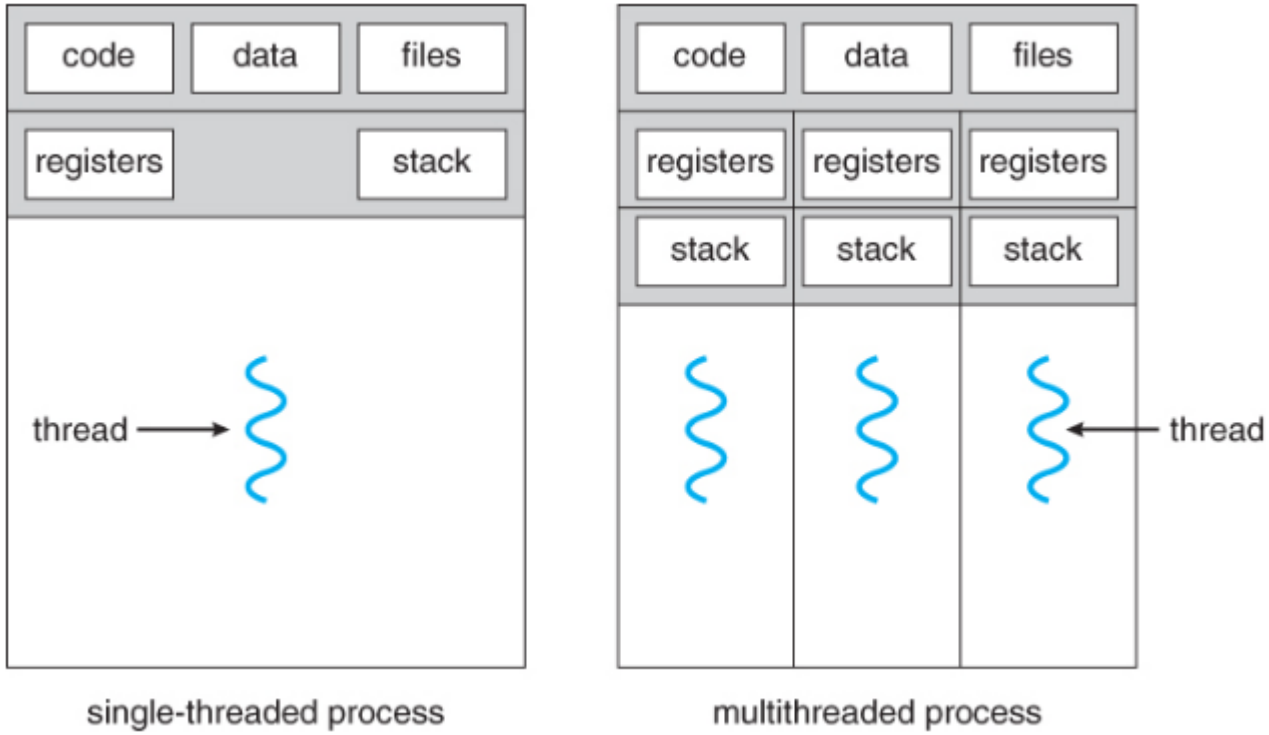
Threads

Threads - תהליכונים

- תהליך אינו יכול לגשת לשום חלק במרחב זיכרון של תהליכים אחרים (אלא אם נגדיר IPC).
- לכל התהליכונים של תהליך מסוים קיימת גישה לאותו מרחב זיכרון.
- ניהול תהליכונים קל יותר ועולה פחות למערכת הפעלה מאשר ניהול תהליכים.

Threads VS fork

- תהליך הנוצר ע"י פעולת FORK – לא חולק שום דבר במשותף עם האבא אלא סה"כ העתק שלו , לעומת תהליכון שחולק מרחב זיכרון, קוד, קבצים משותפים וכו'.
- יצירת תכנית עם תהליכים מרובים היא משימה עתירת משאבים ואילו יצירת תהליכונים היא משימה יחסית קלה למערכת הפעלה.



קטע קריטי

- תהליכונים חולקים אותו אזור בזיכרון.
- ברגע שנפעיל כמה תהליכונים, יתכן כי כמה מהם ינסו לגשת לאותו משתנה בזיכרון -> תוצאות לא צפויות.
- קטע קריטי – אזור בתכנית אשר אסור ששני תהליכונים יהיו בתוכו בו זמנית. במקרים כאלה יש לבצע סינכרון (לדוגמא ע"י סמפורים).

יצירת תכנית עם תהליכונים

- נוסיף לתכנית

```
#include <pthread.h>
```

- קימפול התכנית

```
sysadmin@localhost:~$ gcc thread_program.c -o thread_program -lpthread
```

כאשר :

Thread_program – שם התכנית כולל סיומת

-lpthread ייבוא הקובץ pthread.h

- לבסוף – הרצת התכנית ע"י הפקודה

```
sysadmin@localhost:~$ ./thread_program
```

יצירת תהליכון

```
int pthread_create(pthread_t * thread, pthread_attr_t * attr, void * (*start_routine)(void *), void * arg);
```

- הפונקציה מקבלת –

1. thread – מצביע למקום בזיכרון בו יאוחסן מזהה (id) של התהליכון החדש.
2. attr – מאפיינים המתארים את תכונות התהליכון החדש. בד"כ NULL – תהליכון סטנדרטי עם מאפיינים שהם ברירת מחדל.
3. start_routine – מצביע לפונקציה/קוד שאותה התהליכון יצטרך לבצע.
4. args – פרמטרים שיעברו לפונקציה start_routine בעת הפעלתה.

- הפונקציה מחזירה – 0 במקרה של הצלחה. כמו כן מוכנס מזהה id למצביע thread*

  כל ערך אחר שיחזור – מעיד על כישלון ביצירת התהליכון.

יצירת תהליכון

- דוגמא -

```
nclude <stdio.h>
nclude <stdlib.h>
nclude <unistd.h>
nclude <pthread.h>
id *thread_function(void *args)

  sleep(1);
  printf("Printing this from Thread \n");
  return NULL;

t main()

  pthread_t thread_id;
  printf("Before Thread\n");
  pthread_create(&thread_id, NULL, thread_function, NULL);
  pthread_join(thread_id, NULL);
  printf("After Thread , the pid of thread is: %ld" , thread_id);
  exit(0);
```

```
sysadmin@localhost:~$ gcc thread.c -o thr
sysadmin@localhost:~$ ./thread
Before Thread
Printing this from Thread
After Thread , the pid of thread is: 1403
```

סיום תהליכון

```
void pthread_exit(void *retval);
```

- התהליכון שרץ מסיים את פעולתו, ערך סיום שלו יכנס לתוך מצביע retval.
- תהליך ראשי/שאר תהליכונים יוכלו לראות את ערכו מתוך המצביע הנ"ל.

- דוגמאות שימוש -

```
ret1  = 100;
pthread_exit(&ret1);
```

```
pthread_exit(NULL);
```

המתנה לתהליכון מסוים

```
int pthread_join(pthread_t thread, void **retval);
```

- תהליכון ראשי ממתין לסיומו של תהליכון עם מזהה (id) מסוים.
- הפונקציה מקבלת –

1. thread – מזהה תהליכון שממתינים לסיומו.
2. retval – מצביע למקום בזיכרון בו יאוחסן ערך סיום התהליכון שממתינים לו. (אפשר לסמן בNULL אם אין צורך בערך חזרה).

- הפונקציה מחזירה – במקרה של הצלחה – 0.

כמו כן – בהצלחה – מוכנס ערך סיום ב retval.

כל ערך אחר שיחזור מעיד על כישלון.

```
ad_create(&thread_id, NULL, thread_function, NULL);
ad_join(thread_id, NULL);
```

pthread_self

```
pthread_t pthread_self(void);
```

- pthread_self() - מחזירה מספר מזהה של התהליכון.(בדומה ל getpid() בתהליכים).
- דוגמא לשימוש -

```c
nclude <stdio.h>
nclude <stdlib.h>
nclude <unistd.h>
nclude <pthread.h>
id *ThreadFun(void *vargp)

  sleep(1);
  pthread_t pid_th1 = pthread_self();
  printf("hello , i am thread! my pid is: %ld \n" , pid_th1);
  return NULL;

t main()

  pthread_t thread_id;
  printf("Before Thread\n");
  pthread_create(&thread_id, NULL, ThreadFun, NULL);
  pthread_join(thread_id, NULL);
  printf("After Thread\n");
  exit(0);
```

```
sysadmin@localhost:~$ nano lab1.c
sysadmin@localhost:~$ gcc lab1.c -o lab1 -lpthread
sysadmin@localhost:~$ ./lab1
Before Thread
hello , i am thread! my pid is: 139760804214528
After Thread
```

pthread_equal

```
int pthread_equal(pthread_t t1, pthread_t t2);
```

- פונקציה מקבלת 2 מזהים של תהליכונים.
- מחזירה ערך כלשהו אם המזהים זהים. שונים – מחזירה 0.
- דוגמא –

```
include <pthread.h>
thread_t tmp_thread;
oid* func_one(void* ptr)

rintf(" tmp_thread id is: %ld\n pthread_self() is %ld",tmp_thread,pthread_self
    if (pthread_equal(tmp_thread, pthread_self()) != 0) {
        printf("equal\n");
    } else {
        printf("not equal\n");
    }

nt main()

    pthread_create(&tmp_thread, NULL, func_one, NULL);
    pthread_join(tmp_thread, NULL);
```

```
sysadmin@localhost:~$ gcc lab2.c -o lab2 -lpthread
sysadmin@localhost:~$ ./lab2
 tmp_thread id is: 139729156429568
 , pthread_self() is 139729156429568
equal
```

תהליכונים

- שאלה – מה הפלט ?

```
#include <stdio.h>
#include <pthread.h>
#define NTHREADS 20

void *thread(void *vargp)
{
      static int cnt = 0; //
      cnt++;
      printf("%d\n", cnt);
}

int main ()
{
  int i;
  pthread_t tid;
  for(i = 0; i < NTHREADS; i++)
  {
        pthread_create(&tid, NULL, thread, NULL);
  }
  pthread_exit(NULL);
}
```

תהליכונים

- שאלה – מה הפלט ?

```
#include <stdio.h>
#include <pthread.h>
pthread_t NewThread;
/*-----------------------------------------------*/
void *StartHere(void *Message)
{

    if (pthread_equal(NewThread,pthread_self()))
     {
        printf("%s\n",(char *)Message);
    }
    return(NULL);
}
/*-----------------------------------------------*/
int main(int argc,char *argv[])
{
    pthread_create(&NewThread,NULL,StartHere,"Hello There");

    StartHere("Hello");
    StartHere("There");
    pthread_cond_wait(NewThread,NULL);
}
```

1. Hello There

2. Hello There
   Hello
   There

3. Hello
   Hello there
   There

תהליכונים

- תרגיל – כתוב תכנית שיוצרת 2 תהליכונים.

כל תהליכון מדפיס הודעה.

תהליכון 1 ידפיס – thread1

תהליכון 2 ידפיס – thread2

תהליכונים

- פתרון –

מה יקרה ללא ה join ?

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
void *print_message_function( void *ptr );

main()
{
     pthread_t thread1, thread2;
     char *message1 = "Thread 1";
     char *message2 = "Thread 2";
     pthread_create( &thread1, NULL, print_message_function, (void*) message1);
     pthread_create( &thread2, NULL, print_message_function, (void*) message2);
     pthread_join( thread1, NULL);
     pthread_join( thread2, NULL);
     exit(0);
}

void *print_message_function( void *ptr )
{
     char *message;
     message = (char *) ptr;
     printf("%s \n", message);
}
```

סנכרון בין תהליכונים

- כאשר אנו עובדים עם threads אזי אין צורך במנגנון של IPC כי אין כמה תהליכים. במקום זה הספריה pthread כוללת פקודות של מנגנון mutex בתוכו.

- הרעיון הוא ש thread אחד יכול ל"נעול" קטע קריטי, ואם תהליך אחר ינסה לנעול קטע נעול זה , הוא ייחסם עד שה thread בעל המנגנון יפתח אותו. כך רק תהליך אחד בו-זמנית יכול לעבוד מנגנון

סנכרון בין תהליכונים

מהלך האירועים הטיפוסי הוא כדלהלן:

1. יוצרים ומאתחלים mutex בעל התכונות הדרושות
2. thread או הרבה threads מנסים לנעול את הmutex lock אבל רק אחד יצליח לנעול אותו ואז הmutex שייך לו
3. הthreads האחרים יהיו חסומים
4. ניתן לבדוק אם הmutex פנוי בלי להסתכן בחסימה ע"י פקודת trylock
5. הthread שנכנס עושה משהו (מאחורי הדלת הנעולה)
6. הthread בעל הmutex משחרר אותו
7. thread אחר הופך לבעלים וכל התהליך חוזר חלילה
8. בסוף הmutex יימחק סופית

יצירה ואתחול

- הצהרה על mutex –

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

- הmutex הוא מסוג pthread_mutex_t וחייב להצהיר עליו.
- הדגל PTHREAD_MUTEX_INITIALIZER מצהיר על אתחול ויוצר mutex משוחרר (לא נעול) מוכן לשימוש.

- פונקציית אתחול –

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *mutexattr);
```

- מקבלת:

```
hread_mutex_init(&myMutex, NULL)
```

1. mutex - המנעול עליו נבצע את פעולות הסנכרון.
2. mutexattr – ברירת מחדל NULL .

יש אופציה להגדיר תכונות של mutex (איך יתנהג שנעילה/שחרור וכו'.)

- מחזירה: 0 בהצלחה , כל ערך אחר – כישלון.

פעולות על mutex

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

- נעילת mutex –lock - פעולה זו תחסום את התהליכון שיבצע אותה ובמידה והmutex נעול ע"י תהליכון אחר. ברגע שהmutex יתפנה , התהליכון יכנס פנימה.

- ניסיון לנעילת mutex –trylock - פעולה נכשלת אם הmutex נעול ע"י תהליכון אחר. אם פנוי –> מכניסה פנימה את התהליכון ונועלת את המנעול.

- שחרור mutex –unlock - משחררת את הmutex כדי שתהליכונים אחרים יוכלו לנעול אותו בעתיד.

- מחיקת mutex – destroy – מחיקה/שחרור זיכרון. הפעולה תיכשל אם הmutex עדיין נעול ע"י תהליכון כלשהו.

```c
#include <pthread.h>
pthread_t tid[2];
int counter;
pthread_mutex_t myMutex; // הצהרה על המנעול

void *threadFunc() //פונקציה שמופעלת כשתהליכון נוצר
{
    pthread_mutex_lock(&myMutex); //תהליכון נועל מנעול ונכנס פנימה
    //---   קטע קריטי ----///
    counter += 1;
    printf("\n Job %d has started\n", counter);
    sleep(3);
    printf("\n Job %d has finished\n", counter);
     //---   קטע קריטי ----///
    pthread_mutex_unlock(&myMutex); //תהליכון משחרר מנעול ויוצא מהקטע קריטי
    return NULL;
}

int main(void)
{
    int i = 0;
    if (pthread_mutex_init(&myMutex, NULL) != 0) { //אתחול מנעול
        printf("\n mutex init has failed\n");
        return 1;
    }

    while (i < 2) {
        pthread_create(&(tid[i]),NULL,&threadFunc, NULL); //תהליכון נוצר
        i++;
    }
    pthread_join(tid[0], NULL); //תהליך ראשי ממתין ל2 תהליכונים שיסיימו ריצה
    pthread_join(tid[1], NULL);
    pthread_mutex_destroy(&myMutex);//שחרור זיכרון
    return 0;
}
```

- דוגמא –
- פלט -

```
Job 1 has started

Job 1 has finished

Job 2 has started

Job 2 has finished
```

- מה היה קורה אם הינו מורידים את הנעילה/שחרור מנעול?

משתני תנאי

- מאפשר לתהליכון להמתין לאירוע שיוגדר ע"י המתכנת.
- משתנה תנאי תמיד יעבור עם mutex.

- לדוגמא – נניח 2 תהליכונים מעלים ערך של counter

התהליכון השלישי יתחיל לפעול <u>בתנאי</u> שערך ה counter הגיע ל10.

אתחול ומחיקת משתני תנאי

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
```

```
int pthread_cond_init(pthread_cond_t *cond,
    const pthread_condattr_t *attr);
```

- פונקציית אתחול מקבלת:

1. cond – משתנה תנאי איתו נעבוד.
2. attr – תכונות משתנה תנאי. בתהליכונים אין משמעות לערך , ברירת מחדל – NULL

```
int pthread_cond_destroy(pthread_cond_t *cond);
```

- פונקציית מחיקה מקבלת:

1. cond – משתנה תנאי אותו נרצה למחוק בעת סיום עבודה.

פעולות על משתני תנאי

- המתנה על המשתנה תנאי – `int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);`

1. התהליכון שממתין לאירוע כלשהו , חייב להחזיק בmutex .
2. הפעולה משחררת את הmutex ומעבירה את התהליכון להמתין על המשתנה תנאי.
3. פעולת wait = "מרדימה תהליכון" ומשחררת את הmutex. ברגע שמישהו "מעיר" את התהליכון ממצב המתנה , נועל שוב את ה mutex.

```
int pthread_cond_signal(pthread_cond_t *cond);
```

- שחרור משתנה תנאי –

1. אם קיימים תהליכונים הממתינים על המשתנה תנאי , אחד מהם משתחרר מהתור ועובר להמתין על הmutex.
2. פעולת signal= "מעירה תהליכון"

פעולות על משתני תנאי

- שחרור כל התהליכונים הממתינים על משתנה תנאי –

```
int pthread_cond_broadcast(pthread_cond_t *cond);
```

1. אם קיימים תהליכונים הממתינים על המשתנה – כל התהליכונים מפסיקים להמתין על המשתנה תנאי ועוברים להמתין על ה mutex .
2. התהליכונים יחזרו לפעילות זה אחר זה לאחר שינעלו כל אחד בתורו את המutex .

דוגמא -

```
int    is_done;
mutex_t done_mutex
cond_t   done_cond;
```

```
// Thread A
//do work
mutex_lock(&done_mutex);
  is_done=1;
  cond_signal(&done_cond);
mutex_unlock(&done_mutex);
```

```
// Thread B
//stop until work is done
mutex_lock(&done_mutex);
 if(!is_done)
    cond_wait(&done_cond, &done_mutex);
mutex_unlock(&done_mutex)
```

דוגמא נוספת

```
pthread_mutex_t count_lock;  // mutex  משתנה
pthread_cond_t count_nonzero;  //  משתנה תנאי
unsigned count;
 ... //  אתחול משתנים
decrement_count()  // count - הורדת ערך ה-
{                                          *חובה לנעול לפני
    pthread_mutex_lock(&count_lock);  // mutex  נעילת      נעילת תנאי
    while (count == 0)
        pthread_cond_wait(&count_nonzero, &count_lock);
    count = count - 1;
    pthread_mutex_unlock(&count_lock);
}

increment_count()  //  count  מעלה ערך
{
    pthread_mutex_lock(&count_lock);  // mutex  נעילת
    if (count == 0)  //  אם ערך אפס, נעלה ל-1 ונעיר לממתין
        pthread_cond_signal(&count_nonzero);  //  מקבלת לממתין
    count = count + 1;  //  העלאה ל-1
    pthread_mutex_unlock(&count_lock);  // mutex  שחרור
}
```

דוגמא

```c
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int done = 1;

void *thread_func()
{
    // acquire a lock
    pthread_mutex_lock(&lock);
    if (done == 1) {

        // let's wait on conition variable cond1
        done = 2;
        printf("Waiting on condition variable cond1\n");
        pthread_cond_wait(&cond1, &lock);
    }
    else {

        // Let's signal condition variable cond1
        printf("Signaling condition variable cond1\n");
        pthread_cond_signal(&cond1);
    }
    // release lock
    pthread_mutex_unlock(&lock);
    printf("Returning thread\n");
    return NULL;
}

int main()
{
    pthread_t tid1, tid2;
    pthread_create(&tid1, NULL, thread_func, NULL);
    sleep(2);
    pthread_create(&tid2, NULL, thread_func, NULL);
    pthread_join(tid2, NULL);
    return 0;
}
```

```
Waiting on condition variable cond1
Signaling condition variable cond1
Returning thread
Returning thread
```

מטלה - תהליכונים

- תרגיל 1 - כתוב תכנית שיוצרת תהליכון - מעבירה לפונקציה start_routine פרמטר – מספר שלם כלשהו.

התהליכון מדפיס את המילה HELLO ככמות הפעמים של המספר שעבר.

לדוגמא – העברנו לפונקציה מספר שלם 3 – הפלט יהיה

hello hello hello

```
sysadmin@localhost:~$ ./thread
the number is: 3
hello
hello
hello
```

מטלה - תהליכונים

- תרגיל 2 – כתוב תכנית שיוצרת 10 תהליכונים , כל תהליכון מפעיל פונקציה בעת יצירתו , אליה הוא שולח את האינדקס שלו. בקטע הקריטי מתבצעת סכימה של האינדקסים של כל התהליכונים , הערך הסופי של הסכימה יודפס בסוף. יש להשתמש בmutex שנועל את הקטע קריטי.

- לדוגמא – יצרתי 3 תהליכונים
- אינדקסים : תהליכון ראשון i=1
- תהליכון שני i=2
- תהליכון שלישי i=3

- הקוד ידפיס מספר כלשהו בין 0 ל 1+2+3

מטלה – משתני תנאי

- תרגיל 3 – יש להריץ את הקוד condvar.c
- להבין מה הקוד עושה.

```
  /* For portability, explicitly create threads in a joinable state */
  pthread_attr_init(&attr);
  pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);
//create 3 threads
  pthread_create(&threads[0], &attr, watch_count, (void *)t1);
```

- דגשים על הקוד –
- פונקציה pthread_attr_init(&attr) – מאתחלת תכונות של תהליכון
- פונקציה pthread_attr_setdetachstate– דרך פונקציה זו נשלח דגל שמאפיין תכונה של תהליכונים לדוגמא דגל – PTHREAD_CREATE_JOINABLE

תכונה שניתן להמתין לתהליכון זה.